Assignment 2

CSI 4107

March 15, 2026

Group Members and Tasks:

Caleb Bailey 300281841
Task: Model 1 (SBERT Rerank)

Martin Patrouchev 300286634
Task: Model 2 (Cross-Encoder)

Andrew Pham 300226985
Task: Part 3 (Evaluations)

Lucas Gavura 300310069
Task: Part 4 (Report)

Functionality:

Model 1: SBERT Rerank

Algorithm/flow
This first model uses SBERT to compute the semantic similarity between queries and documents. The system will first perform the initial document retrieval and get the tf-idf score, and then we will compute the cosine similarity and take the top 100 documents based on this score. We then use the SequenceTransformer model to generate the vector representations as follows: each document is encoded into a vector, and the query text is also encoded into the vector. We then compute the cosine similarity score and re-rank the documents based on this new score instead.

Data structures

- Lists
- Arrays
- Dictionary
- Tuples
- Vectors
- Sets

Optimizations:

- Document embedding is precomputed to prevent redundant calculations
- Only the top 100 documents retrieved are passed to the neural re-ranker in a two-stage retrieval
- Precomputing document embeddings to save time

Model 2: Cross-Encoder

Algorithm/flow
This retrieval model uses a Cross-Encoder Architecture. Like our first model, we first do retrieval with the tf-idf cosine similarity model and take the top 100 candidates for neural reranking. Unlike the first model however this one will process the query and document together in a single input. Our system uses the pretrained model: "cross-encoder/ms-marco-TinyBERT-L-2-v2" for our ranking. For every candidate document we pair it together in the form of [query, document] and then process it using our pretrained model for their relevance score. We then use the score to re-rank our documents.
Data structures

- Dictionary
- Array
- Tuples
- Vectors
- Sets

Optimizations
- Using a pretrained model to reduce computational costs, time, and accuracy
- Reduce unnecessary computation by skipping documents with zero vector magnitude

How to Run The Program:

- Install all required packages using “pip install -r requirements.txt”
- eval.py runs all the required scripts, so simply run the following script after running the pip command with the following: “python eval.py” to produce any required output for the assignment

Query Results:
Model 1:
1 Q0 10786948 1 0.291616 SBERT
1 Q0 17388232 2 0.290906 SBERT

1 Q0 4784069 3 0.235627 SBERT
1 Q0 21456232 4 0.224520 SBERT
1 Q0 8891333 5 0.218344 SBERT
1 Q0 16532419 6 0.213598 SBERT
1 Q0 43385013 7 0.205183 SBERT
1 Q0 6308416 8 0.190823 SBERT
1 Q0 36637129 9 0.181613 SBERT
1 Q0 10931595 10 0.181328 SBERT

3 Q0 2739854 1 0.648120 SBERT
3 Q0 23389795 2 0.641222 SBERT
3 Q0 14717500 3 0.601885 SBERT
3 Q0 19058822 4 0.599429 SBERT
3 Q0 15153602 5 0.595896 SBERT
3 Q0 13914198 6 0.564120 SBERT
3 Q0 4632921 7 0.559781 SBERT
3 Q0 32181055 8 0.557863 SBERT
3 Q0 13519661 9 0.557014 SBERT
3 Q0 1388704 10 0.553638 SBERT

Eval summary for Model1 (SBERT rerank)
MAP: 0.6144
P@10: 0.0900

Model 2:
1 Q0 1065627 1 -7.156899 CrossEncoder
1 Q0 43385013 2 -8.731594 CrossEncoder
1 Q0 42240424 3 -9.211911 CrossEncoder
1 Q0 10906636 4 -9.650406 CrossEncoder
1 Q0 10582939 5 -9.683231 CrossEncoder
1 Q0 17518195 6 -9.796329 CrossEncoder
1 Q0 16532419 7 -9.829742 CrossEncoder
1 Q0 36637129 8 -9.870011 CrossEncoder
1 Q0 19651306 9 -9.937504 CrossEncoder
1 Q0 11390393 10 -9.988716 CrossEncoder

3 Q0 14717500 1 1.840725 CrossEncoder
3 Q0 4414547 2 0.700732 CrossEncoder
3 Q0 4632921 3 -0.000821 CrossEncoder
3 Q0 2739854 4 -1.060282 CrossEncoder

3 Q0 20280410 5 -1.555685 CrossEncoder
3 Q0 3672261 6 -1.684777 CrossEncoder
3 Q0 13519661 7 -2.036251 CrossEncoder
3 Q0 23389795 8 -2.195544 CrossEncoder
3 Q0 461550 9 -2.756542 CrossEncoder
3 Q0 13373629 10 -2.790320 CrossEncoder

Eval summary for Model2 (Cross-Encoder)
MAP: 0.6312
P@10: 0.0893

Results:

Query 1

For Query 1, the SBERT model produced similarity scores ranging from approximately 0.18 to 0.29 for the top 10 retrieved documents. The highest-ranked document was 10786948 with a similarity score of 0.2916. The scores decrease very gradually across the ranking, indicating SBERT found several documents with relatively similar semantic similarity to the query in a cluster of sorts.

The CrossEncoder model produced a different ranking. The top document was 1065627, which did not appear for SBERT. Additionally, the CrossEncoder scores are negative for most documents, due to CrossEncoder outputting raw scores rather than their normalized similarity values. Additionally, document 43385013 appears in both rankings but is ranked 7th by SBERT and 2nd by the CrossEncoder; this indicates that 43385013 has more nuanced semantic relevance that only CrossEncoder was able to fully realize.

Query 3

For Query 3, the SBERT model produced significantly higher similarity scores than with Query 1, with the top score reaching 0.6481 for document 2739854. This suggests that SBERT found stronger semantic matches between this query and the previous one.

CrossEncoder shows a different ordering of documents. Interestingly, it identified document 4414547 as the second most relevant result, while this document does not appear in the SBERT top 10 list. This indicates the CrossEncoder model can capture more subtle contextual relationships that are not fully represented with the SBERT embedding space.

Comparison

Comparing the two approaches, the CrossEncoder model generally performed slightly better than the SBERT model. While SBERT had a good MAP score of 0.6144, Cross-Encoder had a better score of 0.6312. However, SBERT did have a better P@10 score at 0.0900 compared to Cross-Encoder's 0.0893. This difference in MAP score is greater than their difference in P@10, so it is better to use Cross-Encoder's ranking over SBERT to ensure overall quality of the system as opposed to a single digit percentage difference on their P@10 scores.

When compared to the baseline system from Assignment 1, which achieved a MAP score of 0.5400, both neural re-ranking methods provide additional semantic understanding but were able to easily outperform the classical retrieval model from our previous assignment.